

МИНОБРНАУКИ РОССИИ
САНКТ-ПЕТЕРБУРГСКИЙ ГОСУДАРСТВЕННЫЙ
ЭЛЕКТРОТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ
«ЛЭТИ» ИМ. В.И. УЛЬЯНОВА (ЛЕНИНА)
Кафедра МО ЭВМ

ОТЧЕТ
по лабораторной работе №4
по дисциплине «Параллельные алгоритмы»
Тема: Параллельное умножение матриц

Студент гр. 1304

Шаврин А.П.

Преподаватель

Сергеева Е.И.

Санкт-Петербург

2024

Цель работы.

Изучить умножения матриц с помощью блочного алгоритма и алгоритма Штрассена и сравнить их производительность.

Задание.

1 Реализовать параллельный алгоритм умножения матриц с блочным разбиением по потокам.

Исследовать масштабируемость выполненной реализации, сравнить с реализацией из работы 1.

2 Реализовать параллельный алгоритм “быстрого” умножения матриц (Штрассена или его модификации).

Тестирование: проверить, что результаты вычислений реализаций 1 и 2 совпадают (в том числе на больших размерностях).

Сравнить производительность с реализацией 1 на больших размерностях данных (порядка $10^4 - 10^6$)

Дополнительно (на отл.) реализация 2 должна быть масштабируема (работать на разном количестве потоков)

Выполнение работы.

1. Были написаны файлы *matrix.hpp* и *matrix.cpp* в которых определены функции:

`std::vector<int> generateMatrix(const int size, const int maxValue)` – генерация матриц заданной размерности

`void writeMatrixInFile(const std::string& filename, const std::vector<int>& result, const int size)` - запись матрицы в файл

`void printMatrix(const std::vector<int>& result, const int size)` – вывод матрицы в консоль

`std::vector<int> matricesMultiplication(const std::vector<int>& A, const std::vector<int>& B, const int size)` – простое перемножение матриц

`std::vector<int> matricesAddition(const std::vector<int>& A, const std::vector<int>& B, const int size)` – сложение матриц

`std::vector<int> matricesSubtraction(const std::vector<int>& A, const std::vector<int>& B, const int size)` – вычитание матриц

2. После были созданы файлы *blocksMultiplication.hpp* и *blocksMultiplication.cpp* в которых описаны функции для блочного перемножения матриц:

`void thMultiplyMatrixBlock(const std::vector<int>& matrixA, const std::vector<int>& matrixB, std::vector<int>& result, std::vector<std::pair<int, int>> blockPositions, const int size)` – функция потока вычисления блоков

`std::vector<int> multiplyMatricesByBlocks(const std::vector<int>& matrixA, const std::vector<int>& matrixB, const int size, const int threadsAmount)` – функция блочного перемножения матриц

В реализации параллельного умножения матриц общее количество элементов финальной матрицы делится на блоки равного размера между N потоками. Если невозможно нацело разделить количество умножений, остаток операций выполняется в последнем потоке. С помощью этого достигался баланс загруженности каждого потока и равномерное вычисление.

3. Затем реализованы функции необходимые для алгоритма Штрассена в файлах *strassenMultiplication.hpp* и *strassenMultiplication.cpp*:

`std::vector<int> multiplyMatricesByStrassen(const std::vector<int>& A, const std::vector<int>& B, const int size, const int threadsAmount)` – функция перемножения матриц по алгоритму Штрассена

`void splitMatrix(const std::vector<int>& A, std::vector<int>& A11, std::vector<int>& A12, std::vector<int>& A21, std::vector<int>& A22, const int size)` - функция разбиения матрицы на 4 подматрицы

`std::vector<int> concatMatrices(const std::vector<int>& C11, const std::vector<int>& C12, const std::vector<int>& C21, const std::vector<int>& C22, const int resultSize)` – функция собирающая результат умножения из 4х подматриц в 1 матрицу.

`void diversificationTasksByThreads(`

```

const int numThreads,
std::vector<int>& D, std::vector<int>& D1,
std::vector<int>& D2, std::vector<int>& H1,
std::vector<int>& H2, std::vector<int>& V1,
std::vector<int>& V2,
const std::vector<int>& A11, const std::vector<int>& A12,
const std::vector<int>& A21, const std::vector<int>& A22,
const std::vector<int>& B11, const std::vector<int>& B12,
const std::vector<int>& B21, const std::vector<int>& B22,
const int size

```

) – функция, распределяющая задачи по потокам

Рекурсивный алгоритм Штрассена при матрице размера 32x32 переходит к обычному умножению матриц, с целью оптимизации его работы.

Функция распределения задач по потокам делит 7 задач на первом уровне рекурсии поровну на переданное количество потоков. С помощью этого достигался баланс загруженности каждого потока и равномерное вычисление. Таким образом есть возможность распараллелить независимые ветви рекурсии. На остальных уровнях рекурсии вычисления происходят последовательно.

4. Для сравнения результатов была написана функция проверки совпадения содержимых файлов в файлах *checker.hpp* и *checker.cpp*:

```

bool isResultMatches(const std::string& filename1, const std::string&
filename2) – функция посимвольного сравнения файлов

```

5. В файле *main.cpp* реализованы функции:

```

int main(int argc, char* argv[]) – принимающая первым параметром размер
матрицы и вторым количество потоков

```

```

void testMatrixMultiplication(const int size, const int threadsCount) – функция
сравнения 2х алгоритмов

```

6. Масштабируемость блочного алгоритма.

Ускорение: $S(p) = T_1 / T_p$, где T_1 — время выполнения на одном потоке, а T_p — на p потоках.

По таблице 2 вычислим Ускорения для каждого количества потоков.

Количество потоков	Ускорение
1	1
2	1.66
3	2.43
4	3.07
5	3.42
6	3.94
7	4.14
8	4.13
9	4.11
10	4.42
11	4.55
12	5.07
15	4.86
20	4.59
30	4.35

Ускорение растет почти линейно при увеличении числа потоков, пока число потоков p меньше числа доступных физических ядер.

С ростом числа потоков сверх числа ядер (12) $S(p)$ уменьшается из-за накладных расходов на переключение контекста.

7. Сравнение производительности.

По таблице 2 можно видеть, что на очень малых размерах матрицы <32 алгоритм Штрассена работает быстрее в силу оптимизации (использование обычного умножения). При размерах матриц в пределах $32 - 512$ по производительности выигрывает алгоритм блочного умножения, однако уже с размеров 512 алгоритм Штрассена начинает превосходить блочный алгоритм по производительности.

По таблице 3 можно видеть, что на всех этапах алгоритм Штрассена выигрывает в производительности блочный алгоритм, однако после 7 потоков скорость алгоритма Штрассена перестает меняться, что связано с реализацией многопоточности в данном алгоритме, в то время как блочный алгоритм продолжает увеличивать производительность до момента максимального доступного количества ядер.

Результаты тестирования см. в приложении А.

Выводы.

В ходе работы была изучена и реализована потокобезопасная структура данных без блокировок, используя шаблон «производитель-потребитель».

В ходе работы были изучены и реализованы блочный алгоритм умножения матриц и алгоритм Штрассена. Была исследована масштабируемость блочного алгоритма и произведено сравнение производительности реализованных алгоритмов без распараллеливания и с распараллеливанием.

В результате исследования масштабируемости блочного алгоритма выявлено, что использование потоков большее чем доступно, не дает выигрыша в производительности.

В результате сравнения было установлено, что алгоритм Штрассена эффективен при больших размерах матриц (размерностью более 512) и неэффективен при малых, т.к. использует большое количество копирований.

ПРИЛОЖЕНИЕ А

ТЕСТИРОВАНИЕ

Таблица 1 – Сравнение производительности алгоритмов умножения матриц в зависимости от размеров матриц (количество потоков 1)

№ п/п	Размеры матриц	Время выполнения в мс для блочного алгоритма	Время выполнения в мс для алгоритма Штрассена
1.	32	0.6922	0.1758
2.	128	9.6360	11.3526
3.	256	66.1122	76.0608
4.	512	651.008	537.968
5.	1024	6693.79	3822.23
6.	2048	146055	27156

Таблица 2 – Сравнение производительности алгоритмов умножения матриц в зависимости от количества потоков (размер матриц 2048)

№ п/п	Количество потоков	Время выполнения в мс для блочного алгоритма	Время выполнения в мс для алгоритма Штрассена
1.	1	146055	27156
2.	2	87808.3	17521
3.	3	59981.6	14365.9
4.	4	47540.6	11468.6
5.	5	42701	11268.9
6.	6	37003.7	10796.4
7.	7	35232.7	6313.36
8.	8	35309.3	6384.97
9.	9	35475.4	5606.93

10.	10	33003.5	5576.12
11.	11	32102.7	6075.71
12.	12	28760.6	5547.31
13.	15	30012.3	6073.5
14.	20	31765.1	5645.31
15.	30	33576.2	6165.11